

Internal Microservice Catalog & Health Portal

Problem Statement #42 | Developer Tools & IT Operations

PES University – Dept. of CSE Lab 1: Requirements Engineering & UML Use-Case ModellingPES University – Dept. of CSE
PES University – Dept. of CSELab 1: Requirements Engineering & UML Use-Case ModellingLab 1: Requirements Engineering
& UML Use-Case Modelling

Internal Microservice Catalog & Health PortalInternal Microservice Catalog & Health PortalInternal Microservice Catalog &
Health Portal

Problem Statement #42 | Developer Tools & IT OperationsProblem Statement #42 | Developer Tools & IT OperationsProblem
Statement #42 | Developer Tools & IT Operations

1. Problem Context & Overview1. Problem Context & Overview1. Problem Context & Overview

An enterprise developer portal for mapping microservice dependencies, aggregating API documentation, and running
automated periodic health checks with downtime alerts.An enterprise developer portal for mapping microservice dependencies,
aggregating API documentation, and running automated periodic health checks with downtime alerts.An enterprise developer
portal for mapping microservice dependencies, aggregating API documentation, and running automated periodic health checks
with downtime alerts.

Target Stakeholders / Actors: DevOps Engineer, System ArchitectTarget Stakeholders / Actors: DevOps Engineer, System
ArchitectTarget Stakeholders / Actors: DevOps Engineer, System Architect

2. Functional Requirements2. Functional Requirements2. Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall ping each registered microservice health endpoint at 30-second intervals.	High	Pass: Health probes are executed approximately every 30 seconds for all registered services. Fail: A registered service is not probed within 40 seconds.	Regular health probes are required to detect service failures quickly.
FR-002	Functional	The system shall calculate and display the rolling 24-hour availability percentage for each microservice.	High	Pass: Availability is calculated correctly from successful and failed health probes over the previous 24 hours. Fail: Incorrect or missing availability data is displayed.	Availability metrics help DevOps engineers evaluate service reliability.
FR-003	Functional	The system shall record and store the result of every health probe including timestamp, status, status code, and latency.	High	Pass: Every probe record contains timestamp, status, status code, and latency. Fail: Any required probe information is missing or incomplete.	Historical probe data is required for availability calculations, troubleshooting, and auditing.
FR-004	Functional	The system shall generate and dispatch an alert when a microservice fails three consecutive health probes.	High	Pass: An alert is generated and dispatched after the third consecutive failed probe. Fail: The alert is missing or generated before three consecutive failures.	Consecutive failures indicate a potentially unavailable service and require operational attention.
FR-005	Functional	The system shall provide searchable API documentation for all registered microservices.	Medium	Pass: API documentation can be searched and the documentation displayed is current. Fail: Documentation is missing, inaccessible, or outdated.	Centralized API documentation helps developers understand and integrate with internal services.

3. Non-Functional Requirements3. Non-Functional Requirements3. Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance	The service catalog dependency graph viewer shall render interactive node architectures containing up to 200 services smoothly.	High	Pass: A dependency graph containing 200 services renders within 1 second and maintains at least 30 FPS during zoom and pan operations. Fail: Rendering takes more than 1 second or interaction falls below 30 FPS.	The portal must remain usable when visualizing large microservice architectures.

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-002	Security	The system shall protect service information and operational data through secure communication, authentication, authorization, and access control.	High	Pass: All traffic uses HTTPS, users are authenticated, and role-based access controls restrict unauthorized operations. Fail: Unauthenticated users can access protected information or unauthorized users can perform restricted operations.	Microservice information and operational health data may contain sensitive internal infrastructure details.

4. UML Use-Case Diagram4. UML Use-Case Diagram4. UML Use-Case Diagram

The diagram below contains the primary actors, core use cases, and the required <> and <> relationships.The diagram below contains the primary actors, core use cases, and the required <> and <> relationships.The diagram below contains the primary actors, core use cases, and the required <> and <> relationships.

5. Use-Case Flow Specification5. Use-Case Flow Specification5. Use-Case Flow Specification

Use Case ID	UC-001
Use Case Name	Monitor Service Health
Primary Actor	DevOps Engineer
Goal	Monitor microservice health and availability in real time.

5.1 Preconditions5.1 Preconditions5.1 Preconditions

The user is authenticated and has permission to access the portal.The user is authenticated and has permission to access the portal.The user is authenticated and has permission to access the portal.

At least one microservice is registered in the system.At least one microservice is registered in the system.At least one microservice is registered in the system.

The microservice health endpoints are configured and accessible.The microservice health endpoints are configured and accessible.The microservice health endpoints are configured and accessible.

5.2 Main Success Scenario5.2 Main Success Scenario5.2 Main Success Scenario

DevOps Engineer navigates to Monitor Service Health.DevOps Engineer navigates to Monitor Service Health.DevOps Engineer navigates to Monitor Service Health.

The system displays all registered microservices with their current health status.The system displays all registered microservices with their current health status.The system displays all registered microservices with their current health status.

The system sends a health probe to each microservice every 30 seconds.The system sends a health probe to each microservice every 30 seconds.The system sends a health probe to each microservice every 30 seconds.

Each microservice returns a health response.Each microservice returns a health response.Each microservice returns a health response.

The system records the response timestamp, status, status code, and latency.The system records the response timestamp, status, status code, and latency.The system records the response timestamp, status, status code, and latency.

The system calculates the rolling 24-hour availability percentage.The system calculates the rolling 24-hour availability percentage.The system calculates the rolling 24-hour availability percentage.

The system updates the health monitoring dashboard.The system updates the health monitoring dashboard.The system updates the health monitoring dashboard.

The DevOps Engineer views the current health and availability information.The DevOps Engineer views the current health and availability information.The DevOps Engineer views the current health and availability information.

The use case ends successfully.The use case ends successfully.The use case ends successfully.

5.3 Alternate Flow – Alert Generation / Service Failure5.3 Alternate Flow – Alert Generation / Service Failure5.3 Alternate Flow – Alert Generation / Service Failure

The system detects that a microservice has failed a health probe. The system detects that a microservice has failed a health probe. The system detects that a microservice has failed a health probe.

The system records the failed probe. The system records the failed probe. The system records the failed probe.

The system checks whether three consecutive probes have failed. The system checks whether three consecutive probes have failed. The system checks whether three consecutive probes have failed.

If fewer than three consecutive failures have occurred, the system continues normal monitoring. If fewer than three consecutive failures have occurred, the system continues normal monitoring. If fewer than three consecutive failures have occurred, the system continues normal monitoring.

If three consecutive failures have occurred, the system generates a failure alert. If three consecutive failures have occurred, the system generates a failure alert. If three consecutive failures have occurred, the system generates a failure alert.

The system sends the alert through the configured notification service. The system sends the alert through the configured notification service. The system sends the alert through the configured notification service.

The alert is recorded and displayed under Manage Alerts. The alert is recorded and displayed under Manage Alerts. The alert is recorded and displayed under Manage Alerts.

The system returns to the main monitoring flow. The system returns to the main monitoring flow. The system returns to the main monitoring flow.

5.4 Postconditions5.4 Postconditions5.4 Postconditions

The latest health status of all monitored microservices is stored. The latest health status of all monitored microservices is stored. The latest health status of all monitored microservices is stored.

Probe results and availability information are updated. Probe results and availability information are updated. Probe results and availability information are updated.

Any qualifying service failure is recorded as an alert. Any qualifying service failure is recorded as an alert. Any qualifying service failure is recorded as an alert.

The monitoring dashboard reflects the latest service status. The monitoring dashboard reflects the latest service status. The monitoring dashboard reflects the latest service status.

5.5 Success Guarantee5.5 Success Guarantee5.5 Success Guarantee

The system continuously monitors registered microservices and provides accurate health and availability information to authorized users. The system continuously monitors registered microservices and provides accurate health and availability information to authorized users. The system continuously monitors registered microservices and provides accurate health and availability information to authorized users.

4. UML Use-Case Diagram

The diagram below contains the primary actors, core use cases, and the required <<include>> and <<extend>> relationships.

PROBLEM STATEMENT #42 | Developer Tools & IT Operations

Internal Microservice Catalog & Health Portal

1. PROBLEM CONTEXT & OVERVIEW

An enterprise developer portal mapping microservice dependencies, aggregating API documentation, and running automated periodic health check pingers with downtime alerts.

Target Stakeholders / Actors: DevOps Engineer, System Architect

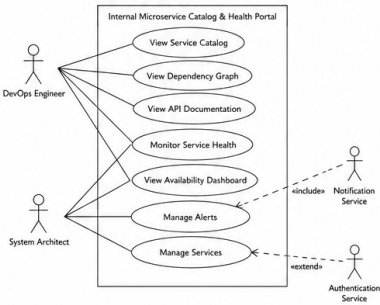
2. FUNCTIONAL REQUIREMENTS (FR)

ID	Description	Priority	Acceptance Criteria
FR-001	The system shall ping each microservice health endpoint at 30-second intervals.	High	Pass: Interval between pings is ~30s (±5s) for all services. Fail: Any service not pinged within 40s.
FR-002	The system shall compute rolling 24-hour service availability percentage for each microservice.	High	Pass: Availability is calculated correctly as (Up pings / Total pings) × 100 over last 24 hours. Fail: Incorrect calculation or missing data.
FR-003	The system shall record and store ping results with timestamp, status code, and latency.	High	Pass: Each ping record has timestamp, status, latency. Fail: Missing or incomplete ping records.
FR-004	The system shall raise an alert when a service is unreachable in 3 consecutive health probes.	High	Pass: Alert generated and dispatched on 3rd consecutive failure. Fail: Alert not generated or generated before 3 failures.
FR-005	The system shall provide API documentation aggregated from all registered microservices.	Medium	Pass: Docs are discoverable, searchable and up to date. Fail: Missing service docs or outdated information.

3. NON-FUNCTIONAL REQUIREMENTS (NFR)

ID	Description	Type	Acceptance Criteria	Rationale
NFR-001	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly.	Performance	Pass: Graph with 200 nodes renders with FPS ≥ 30 and interactions (zoom/pan) respond < 1s. Fail: Rendering lag > 1s or FPS < 30.	Ensure usability for large-scale systems.
NFR-002	The system shall meet security standards for data in transit, access control, and alert privacy.	Security	Pass: All traffic over HTTPS, role-based access enforced, alert data visible only to authorized roles. Fail: Any data leakage or unauthorized access.	Protect system and sensitive operational data.

4. UML USE-CASE DIAGRAM



5. USE-CASE FLOW SPECIFICATION

Use Case ID:	UC-001
Use Case Name:	Monitor Service Health
Primary Actor:	DevOps Engineer
Goal:	Monitor microservice health and availability in real-time.
Preconditions	1. User is authenticated and has access to the portal. 2. At least one microservice is registered in the system.
Main Success Scenario	1. Actor navigates to "Monitor Service Health". 2. System displays the list of all microservices with current status. 3. System pings each service health endpoint every 30 seconds. 4. System records ping result with status, latency, and timestamp. 5. System updates rolling 24-hour availability percentage. 6. Actor views real-time status and availability dashboard. 7. System raises alert upon 3 consecutive failed probes (see Alternate Flow). 8. Use case ends.
Postconditions	• Ping results and availability data are updated in the system. • Dashboard reflects the latest status.
Success Guarantee	System continuously monitors services and provides accurate health status.

6. ALTERNATE FLOW – ALERT GENERATION (EXTENSION TO UC-001)

Trigger:	Service is unreachable in 3 consecutive health probes.
Steps:	1. System detects 3 consecutive failed probes. 2. System creates an alert with service name, timestamp, failure reason. 3. System sends alert notification via Email / Slack / PagerDuty (configured). 4. Alert is logged and visible in "Manage Alerts". 5. Use case returns to Main Success Scenario at Step 6.

5. Use-Case Flow Specification

Use Case ID	UC-001
Use Case Name	Monitor Service Health
Primary Actor	DevOps Engineer
Goal	Monitor microservice health and availability in real time.

5.1 Preconditions

1. The user is authenticated and has permission to access the portal.
2. At least one microservice is registered in the system.
3. The microservice health endpoints are configured and accessible.

5.2 Main Success Scenario

1. DevOps Engineer navigates to Monitor Service Health.
2. The system displays all registered microservices with their current health status.
3. The system sends a health probe to each microservice every 30 seconds.
4. Each microservice returns a health response.
5. The system records the response timestamp, status, status code, and latency.
6. The system calculates the rolling 24-hour availability percentage.
7. The system updates the health monitoring dashboard.
8. The DevOps Engineer views the current health and availability information.
9. The use case ends successfully.

5.3 Alternate Flow – Alert Generation / Service Failure

- A1. The system detects that a microservice has failed a health probe.
- A2. The system records the failed probe.
- A3. The system checks whether three consecutive probes have failed.
- A4. If fewer than three consecutive failures have occurred, the system continues normal monitoring.
- A5. If three consecutive failures have occurred, the system generates a failure alert.
- A6. The system sends the alert through the configured notification service.
- A7. The alert is recorded and displayed under Manage Alerts.
- A8. The system returns to the main monitoring flow.

5.4 Postconditions

- The latest health status of all monitored microservices is stored.
- Probe results and availability information are updated.
- Any qualifying service failure is recorded as an alert.
- The monitoring dashboard reflects the latest service status.

5.5 Success Guarantee

The system continuously monitors registered microservices and provides accurate health and availability information to authorized users.